JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface Annotation

All Known Implementing Classes:

BeanProperty, BooleanFlag, Category, ConstructorParameters, ConstructorProperties, ContentType, Contextual, DataAmount, Deprecated, Description, DescriptorKey, Documented, Enabled, Experimental, Frequency, FunctionalInterface, Generated, Inherited, JavaBean, Label, MemoryAddress, MetadataDefinition, MXBean, Name, Native, Override, Percentage, Period, Registered, Relational, Repeatable, Retention, SafeVarargs, Serial, SettingDefinition, StackTrace, SupportedAnnotationTypes, SupportedOptions, SupportedSourceVersion, SuppressWarnings, SwingContainer, Target, Threshold, Throttle, Timespan, Timestamp, Transient, TransitionFrom, TransitionTo, Unsigned

---

`public interface Annotation`

The common interface extended by all annotation interfaces. Note that an interface that manually extends this one does *not* define an annotation interface. Also note that this interface does not itself define an annotation interface. More information about annotation interfaces can be found in section §9.6 of *The Java Language Specification*. The `AnnotatedElement` interface discusses compatibility concerns when evolving an annotation interface from being non-repeatable to being repeatable.

Since:

1.5

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
`Class<? extends Annotation>`	**`annotationType()`**	Returns the annotation interface of this annotation.
`boolean`	**`equals(Object obj)`**	Returns true if the specified object represents an annotation that is logically equivalent to this one.
`int`	**`hashCode()`**	Returns the hash code of this annotation.

String	**toString**()	Returns a string representation of this annotation.

Method Details

equals

`boolean equals(Object obj)`

Returns true if the specified object represents an annotation that is logically equivalent to this one. In other words, returns true if the specified object is an instance of the same annotation interface as this instance, all of whose members are equal to the corresponding member of this annotation, as defined below:

- Two corresponding primitive typed members whose values are `x` and `y` are considered equal if `x == y`, unless their type is `float` or `double`.
- Two corresponding `float` members whose values are `x` and `y` are considered equal if `Float.valueOf(x).equals(Float.valueOf(y))`. (Unlike the == operator, NaN is considered equal to itself, and `0.0f` unequal to `-0.0f`.)
- Two corresponding `double` members whose values are `x` and `y` are considered equal if `Double.valueOf(x).equals(Double.valueOf(y))`. (Unlike the == operator, NaN is considered equal to itself, and `0.0` unequal to `-0.0`.)
- Two corresponding `String`, `Class`, enum, or annotation typed members whose values are `x` and `y` are considered equal if `x.equals(y)`. (Note that this definition is recursive for annotation typed members.)
- Two corresponding array typed members `x` and `y` are considered equal if `Arrays.equals(x, y)`, for the appropriate overloading of `Arrays.equals`.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the reference object with which to compare.

Returns:

true if the specified object represents an annotation that is logically equivalent to this one, otherwise false

See Also:

`Object.hashCode()`, `HashMap`

hashCode

`int hashCode()`

Returns the hash code of this annotation.

The hash code of an annotation is the sum of the hash codes of its members (including those with default values). The hash code of an annotation member is (127 times the hash code of the member-name as computed by `String.hashCode()`) XOR the hash code of the member-value. The hash code of a member-value depends on its type as defined below:

- The hash code of a primitive value *v* is equal to *WrapperType*`.valueOf(`*v*`).hashCode()`, where *WrapperType* is the wrapper type corresponding to the primitive type of *v* (`Byte`, `Character`, `Double`, `Float`, `Integer`, `Long`, `Short`, or `Boolean`).
- The hash code of a string, enum, class, or annotation member-value *v* is computed as by calling *v*`.hashCode()`. (In the case of annotation member values, this is a recursive definition.)
- The hash code of an array member-value is computed by calling the appropriate overloading of `Arrays.hashCode` on the value. (There is one overloading for each primitive type, and one for object reference types.)

Overrides:

`hashCode` in class `Object`

Returns:

the hash code of this annotation

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(Object)`

toString

`String toString()`

Returns a string representation of this annotation. The details of the representation are implementation-dependent, but the following may be regarded as typical:

```
@com.example.Name(first="Duke", middle="of", last="Java")
```

Overrides:

`toString` in class `Object`

Returns:

a string representation of this annotation

annotationType

`Class<? extends Annotation> annotationType()`

Returns the annotation interface of this annotation.

API Note:

Implementation-dependent classes are used to provide the implementations of annotations. Therefore, calling `getClass` on an annotation will return an implementation-dependent class. In contrast, this method will reliably return the annotation interface of the annotation.

Returns:

the annotation interface of this annotation

See Also:

`Enum.getDeclaringClass()`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.